

SECTION 2 — TUTORIAL SOLUTIONS (Tut-3 → Tut-7)

Scope: every question in Tutorials 3–7, **including** sequential circuits (latches, flip-flops, counters, FSMs, state diagrams), and **excluding** register questions as you asked. Tut-7 Q4 (shift-register delay line) and Tut-7 Q5 (serial adder built on shift registers) are left out for that reason. **Checking:** every K-map answer below was cross-checked by an exhaustive Quine-McCluskey run, and every designed circuit was simulated over all input combinations. **Notation:** ' = complement, $\oplus$ = XOR, $\odot$ = XNOR, $Q+$ = next state. In a state table, a cell like 01/0 means next state 01, output 0.

QUICK REFERENCE — FLIP-FLOP TABLES (used in Tut-6 and Tut-7)

Characteristic	D	T	JK	SR
Equation	$Q+ = D$	$Q+ = T \oplus Q$	$Q+ = JQ' + K'Q$	$Q+ = S + R'Q$ (SR = 11 not allowed)

Excitation tables (what input causes a given transition):

$Q \rightarrow Q+$	D	T	J	K	S	R
$0 \rightarrow 0$	0	0	0	X	0	X
$0 \rightarrow 1$	1	1	1	X	1	0
$1 \rightarrow 0$	0	1	X	1	0	1
$1 \rightarrow 1$	1	0	X	0	X	0

- **D:** $D = Q+$, with no don't-cares. Easiest to fill in, often the most logic.
- **T:** $T = 1$ exactly when the bit **changes**. Ideal for counters.
- **JK:** every row has an X, which gives the most freedom and usually the least logic. Rule of thumb: when $Q = 0$ only J matters ($J = Q+$); when $Q = 1$ only K matters ($K = (Q+)'$).

Sequential design recipe (Lec-20-21):

1. State diagram.
2. State table.
3. Assign binary codes.
4. Excitation table: for each flip-flop, fill in its inputs from the excitation table.
5. K-maps $\rightarrow$ flip-flop input equations and output equation.
6. Draw the circuit.
7. Check the unused states.

TUTORIAL 3 — Subtractors, Adder/Subtractor, Scaling, Reading Verilog

Tut-3 Q1 — Building a Faster Subtractor

(a) Half-subtractor truth table and minimal SOP from first principles

- $X - Y$ for 1-bit X, Y :
 - $0 - 0 = 0$, no borrow.
 - $0 - 1$: we must borrow 1 from the next column (worth 2), so the difference = $2 - 1 = 1$, borrow = 1.
 - $1 - 0 = 1$, no borrow.
 - $1 - 1 = 0$, no borrow.

X	Y	D	B
0	0	0	0
0	1	1	1
1	0	1	0
1	1	0	0

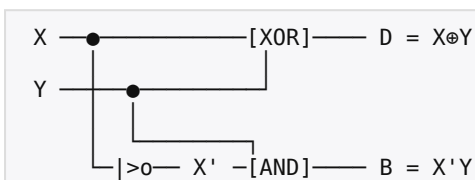
- K-map for D:

X \ Y	0	1
0	0	1
1	1	0

- Checkerboard: the two 1s are diagonal, not adjacent, so no grouping is possible. $D = X'Y + XY'$ $= X \oplus Y$.
- K-map for B:

X \ Y	0	1
0	0	1
1	0	0

- A single isolated 1 $\rightarrow B = X'Y$.



(b) Algebraic proof: full subtractor = 2 half subtractors + OR

- HS1 on (X, Y) : $D1 = X \oplus Y$, $B1 = X'Y$.
- HS2 on $(D1, Bin)$, computing $D1 - Bin$: $D2 = D1 \oplus Bin$, $B2 = D1' \cdot Bin$.
- **Difference:** $D2 = (X \oplus Y) \oplus Bin = X \oplus Y \oplus Bin$ ✓ (the standalone FS difference).
- **Borrow:**
 - $Bout = B1 + B2 = X'Y + (X \oplus Y)' \cdot Bin$
 - $= X'Y + (XY + X'Y') \cdot Bin$ (XNOR expanded)
 - $= X'Y + XY \cdot Bin + X'Y' \cdot Bin$
 - $= X'(Y + Y' \cdot Bin) + XY \cdot Bin$ (factor X' from the 1st and 3rd terms)
 - $= X'(Y + Bin) + XY \cdot Bin$ (redundant literal: $Y + Y' \cdot Bin = Y + Bin$)
 - $= X'Y + X'Bin + XY \cdot Bin$

- $= X'Y + \text{Bin}(X' + XY)$
- $= X'Y + \text{Bin}(X' + Y)$ (redundant literal: $X' + XY = X' + Y$)
- $= X'Y + X'\text{Bin} + Y\cdot\text{Bin}$ ✓ (the standalone FS borrow).

(c) Borrow-generate and borrow-propagate

- Rewrite: $B(i+1) = X_i'Y_i + (X_i' + Y_i) \cdot B_i$, which has the form $G_i + P_i \cdot B_i$.
- $G_i = X_i'Y_i$: this bit position produces a borrow by itself ($X = 0, Y = 1$).
- $P_i = X_i' + Y_i$: this bit passes an incoming borrow on ($X = 0$, or $Y = 1$).
- ⚠ EXAM TRICK: $P_i = X_i \oplus Y_i$ does **not** work here. At $X = 1, Y = 1, B_{in} = 1$: $1 - 1 - 1 = -1$ needs a borrow, but $X_i \oplus Y_i = 0$ would block it. The OR form gives $P = 1$ ✓.

(d) Direct expression for B4

- Unroll $B(i+1) = G_i + P_i \cdot B_i$ four times:
 - $B_1 = G_0 + P_0 \cdot B_{in}$
 - $B_2 = G_1 + P_1G_0 + P_1P_0 \cdot B_{in}$
 - $B_3 = G_2 + P_2G_1 + P_2P_1G_0 + P_2P_1P_0 \cdot B_{in}$
 - $B_4 = G_3 + P_3G_2 + P_3P_2G_1 + P_3P_2P_1G_0 + P_3P_2P_1P_0 \cdot B_{in}$

Key Logic & Takeaway

- The half-subtractor borrow $X'Y$ is asymmetric. That one fact drives Q1(b), Q1(c) and Q3(b).
- Borrow-lookahead has exactly the same structure as carry-lookahead; only G and P change ($X'Y, X' + Y$).

Tut-3 Q2 — A Life-Counter Bug (overflow detective)

(a) Bit-width and the failure condition

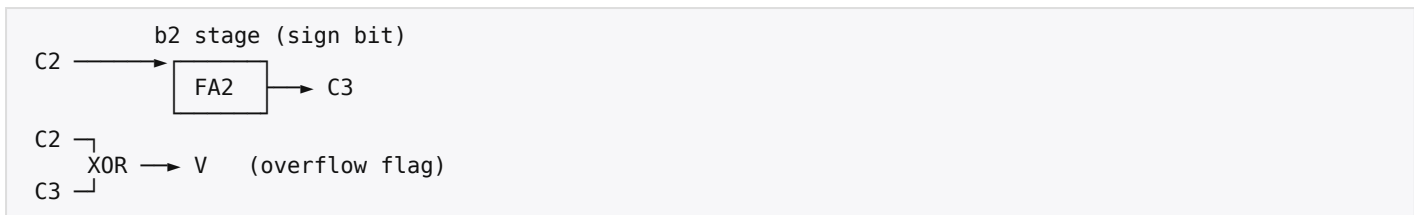
- **Try widths:**
 - 2-bit signed ($-2 \dots +1$): the correct row " $1 + 1 = 2$ " could not be shown → rejected.
 - **3-bit signed ($-4 \dots +3$):**
 - $2 + 2 = 4 \rightarrow$ wraps to $4 - 8 = -4$ ✓ (matches the log)
 - $3 + 1 = 4 \rightarrow -4$ ✓
 - $-3 - 3 = -6 \rightarrow -6 + 8 = 2$ ✓
 - $-4 - 3 = -7 \rightarrow -7 + 8 = 1$ ✓
 - $1 + 1 = 2$ ✓ and $2 - 1 = 1$ ✓
 - 4-bit signed ($-8 \dots +7$): $2 + 2$ would show 4 (correct), but the log shows $-4 \rightarrow$ rejected.
- **The circuit is a 3-bit 2's-complement adder/subtractor.**
- **Failure condition:** overflow happens exactly when the two operands the adder actually adds (A and B for $S = 0$; A and $B' + 1$ for $S = 1$) have the **same sign** and the result has the **opposite sign**. Mixed-sign operands can never overflow.

(b) Overflow flag from existing signals

- $V = C_2 \oplus C_3$ = (carry into the sign bit) XOR (carry out of the sign bit). Generally $V = C_{(n-1)} \oplus C_n$. One XOR gate.
- Bit-level verification (3-bit adder, bits b2 b1 b0, subtraction done as $A + B' + 1$):

Before	Event	Internal addition	C2	C3	Sum bits	Shown	V
1	+1	001 + 001 + 0	0	0	010	2	0
2	+2	010 + 010 + 0	1	0	100	-4	1
3	+1	011 + 001 + 0	1	0	100	-4	1
2	-1	010 + 110 + 1	1	1	001	1	0
-3	-3	101 + 100 + 1	0	1	010	2	1
-4	-3	100 + 100 + 1	0	1	001	1	1

- $V = 1$ on exactly the four wrong rows.



(c) Does the flag need changes for subtraction?

- **No.** In subtract mode the circuit adds A to B' with carry-in 1, so the adder still performs an ordinary n -bit addition.
- $C2$ and $C3$ are the same physical wires in both modes, and the same overflow rule applies to "the two numbers actually added". Rows 5 and 6 ($S = 1$) are flagged correctly.

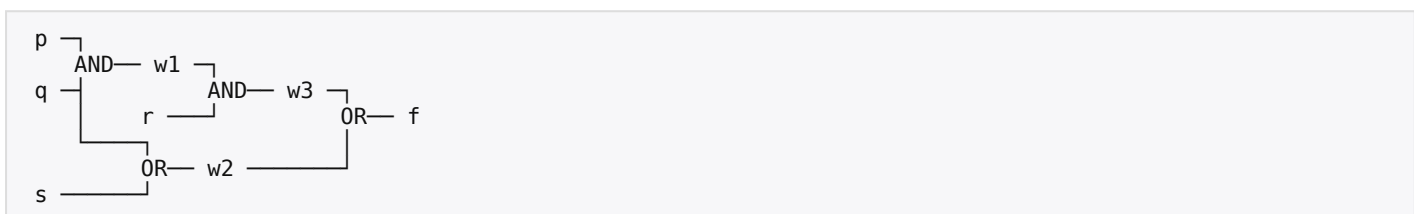
Key Logic & Takeaway

- Bit-width detective: a correct row gives a lower bound; a wrong row gives the modulus (wrong = true $\pm 2^n$).
- $V = C(n-1) \oplus C_n$ works unchanged for add and subtract.
- Signed overflow requires same-sign operands as **seen by the adder** (B is inverted in subtract mode).

Tut-3 Q3 — Reading Circuits, Not Writing Them

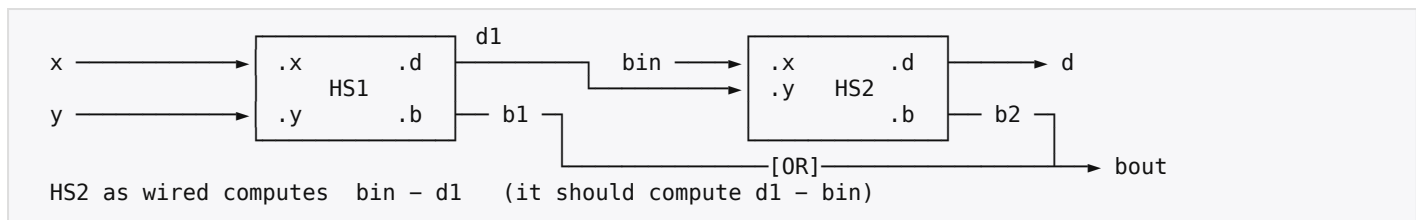
(a) The `mystery` module

- Gate primitives list the output first:
 - `and(w3, w1, r) → w3 = w1 · r`
 - `or(f, w3, w2) → f = w3 + w2`
 - `and(w1, p, q) → w1 = pq`
 - `or(w2, q, s) → w2 = q + s`
- The order of the lines does not matter: Verilog structural statements are concurrent wiring.



- $f = pqr + q + s = q(pr + 1) + s = \mathbf{q + s}$ (absorption).
- p and r have **no effect**: the module is a 2-input OR of q and s .

(b) Full subtractor built from two `half_subtractor` instances



- **d:** HS2 gives $\text{bin} \oplus d1 = x \oplus y \oplus \text{bin} \rightarrow$ **correct for all inputs** (XOR is symmetric).
- **bout:** HS2's borrow as wired is $\text{bin}' \cdot d1$, but the correct second-stage borrow is $d1' \cdot \text{bin}$. **Wrong.**
- **Failing input** $(x, y, \text{bin}) = (0, 0, 1)$:
 - $d1 = 0, b1 = 0$.
 - As wired: $b2 = \text{bin}' \cdot d1 = 0 \cdot 0 = 0 \rightarrow \text{bout} = 0$ **X**.
 - Correct: $0 - 0 - 1 = -1$ needs a borrow $\rightarrow \text{bout} = 1$.
- Full comparison:

x y bin	correct bout	wired bout	
0 0 0	0	0	
0 0 1	1	0	X
0 1 0	1	1	
0 1 1	1	1	
1 0 0	0	1	X ($d1 = 1, \text{bin} = 0 \rightarrow \text{bin}'d1 = 1$)
1 0 1	0	0	
1 1 0	0	0	
1 1 1	1	0	X

- **Wrong connection:** HS2's port mapping. **Fix:** `half_subtractor HS2(.x(d1), .y(bin), .d(d), .b(b2));`

(c) Testbench declarations

```
module tb_full_subtractor;
  wire t_x, t_y, t_bin;    // x assigned in initial → must be reg
  reg t_d, t_bout;         // x driven by DUT outputs → must be wire
  ...
```

- **No, it will not simulate.** All five declarations have the wrong type.
- `t_x, t_y, t_bin` are assigned procedurally inside `initial`. A `wire` is a physical connection with no storage; it only carries what a continuous driver (gate output, `assign`, module output) puts on it, and nothing stores a value "set" at time 0. A signal set procedurally must hold its value until the next assignment, which is exactly what `reg` does. $\rightarrow$ **reg**.
- `t_d, t_bout` are connected to the DUT's **output** ports, which drive them continuously. A `reg` can only be written procedurally; it cannot be driven by a module output. $\rightarrow$ **wire**.
- Corrected:

```

module tb_full_subtractor;
  reg t_x, t_y, t_bin;
  wire t_d, t_bout;
  full_subtractor DUT(.x(t_x), .y(t_y), .bin(t_bin), .d(t_d), .bout(t_bout));
  initial begin
    t_x = 0; t_y = 0; t_bin = 0;
    #5 t_x = 0; t_y = 1; t_bin = 0;
  end
endmodule

```

Key Logic & Takeaway

- Simplify any structural netlist before trusting it: absorption often removes inputs.
- Non-symmetric blocks (subtractors) expose port-order bugs; XOR outputs hide them.
- Testbench: what **you** drive = `reg`; what the **DUT** drives = `wire`.

Tut-3 Q4 — How Do Adders Scale?

(a) Ripple-carry adder

- Gate count: n full adders $\times$ a fixed number of gates each $\rightarrow O(n)$.
- Delay: $2n + 1$ gate levels $\rightarrow O(n)$. Doubling n doubles the delay.

(b) Flat carry-lookahead for n bits

- Small cases:
 - $n = 1$: $C_1 = G_0 + P_0C_0 \rightarrow 2$ product terms.
 - $n = 2$: C_2 has 3 terms; total $2 + 3 = 5$.
 - $n = 4$: $C_1 \dots C_4$ have $2 + 3 + 4 + 5 = 14$ terms.
 - General: C_i has $i + 1$ terms $\rightarrow$ total $\sum(i+1) \approx n^2/2 \rightarrow$ **gates $O(n^2)$** .
- **Delay $O(1)$** : every carry is a flat 2-level AND-OR, so P/G (1) + carries (2) + sum XOR (1) ≈ 4 levels for any n .
- **Practical problem**: the last term of C_n , $P_{(n-1)} \dots P_0 \cdot C_0$, needs an AND gate with $n + 1$ **inputs**. Huge fan-in is not physically realisable, and big gates are slow anyway.

(c) 4-bit CLA blocks chained (block-ripple), $n = 64$

- 16 blocks; each passes its carry to the next after a constant block delay `t_blk` (a few gate levels).
- **Total $\approx (n/4) \times t_{\text{blk}} = 16 \times t_{\text{blk}} \rightarrow$ still $O(n)$** .
- Improved: the **constant factor** (slope about $4\times$ smaller than plain ripple).
- Not changed: the **growth rate**; doubling n still doubles the delay.

(d) Hierarchical lookahead

- Block signals from the 4 bit-level G/P:
 - $G_{\text{blk}} = G_3 + P_3G_2 + P_3P_2G_1 + P_3P_2P_1G_0$ (block generates a carry by itself).
 - $P_{\text{blk}} = P_3P_2P_1P_0$ (block passes an incoming carry straight through).
- A second-level lookahead unit (identical to (b), one level up) computes every block's carry-in directly from `G_blk`, `P_blk`.

Level 0: 64 bits $\rightarrow$ 16 groups of 4 (bit G/P $\rightarrow$ block G/P)
 Level 1: 16 blocks $\rightarrow$ 4 groups of 4 (block G/P $\rightarrow$ super-block G/P)
 Level 2: 4 super-blocks $\rightarrow$ 1 group ($\rightarrow$ C64 and all super-block carries)
 then carries flow back down: level 2 $\rightarrow$ level 1 $\rightarrow$ level 0 $\rightarrow$ sums

- **Delay:** each level adds a constant, and the number of levels is $\log_4 n \rightarrow \mathbf{O(\log n)}$.
- **Gates:** each level has 1/4 as many units as the level below ($16 + 4 + 1 + \dots$), a geometric series $\rightarrow \mathbf{O(n)}$ total.
- **$n = 64$:** $\log_4 64 = 3$ levels. About $3 \times 4 \approx \mathbf{12}$ gate delays, versus **129** for ripple-carry ($\approx 10\times$ faster).

Key Logic & Takeaway

Design	Gates	Delay
Ripple	$O(n)$	$O(n)$
Flat CLA	$O(n^2)$	$O(1)$, but fan-in explodes
Chained 4-bit CLAs	$O(n)$	$O(n)$, smaller slope
Hierarchical CLA	$O(n)$	$O(\log n)$

TUTORIAL 4 — 4-Variable K-maps with Don't-Cares

Tut-4 Q1 — Relationship between MSP and MPS

(a) Map with no don't-cares

AB \ CD	00	01	11	10
00	0	0	1	0
01	0	0	1	1
11	0	0	1	1
10	0	0	1	0

- $F = \sum m(3, 6, 7, 11, 14, 15)$.
- **Grouping the 1s:**
 - CD = whole column 11 = $\{3, 7, 15, 11\} \rightarrow \mathbf{EPI}$ (only PI covering m_3, m_{11}).
 - BC = rows 01, 11 $\times$ columns 11, 10 = $\{7, 6, 15, 14\} \rightarrow \mathbf{EPI}$ (only PI covering m_6, m_{14}).
 - PIs = EPIs = $\{CD, BC\}$. **MSP:** $F = CD + BC$ (4 literals).
- **Grouping the 0s:**
 - C' = columns 00, 01 (8 cells) $\rightarrow \mathbf{EPI}$.
 - $B'D'$ = four corners $\{0, 2, 8, 10\} \rightarrow \mathbf{EPI}$ (only one covering m_2, m_{10}).
 - $F' = C' + B'D' \rightarrow \mathbf{MPS: } F = C(B + D)$.
- **MSP = MPS?** Yes: $C(B + D) = BC + CD$. With no don't-cares, both describe the unique function.

(b) Map with don't-cares

AB \ CD	00	01	11	10
00	X	1	1	X
01	0	X	X	0
11	0	0	0	0
10	1	0	0	1

- 1s: {1, 3, 8, 10}; Xs: {0, 2, 5, 7}; 0s: {4, 6, 9, 11, 12, 13, 14, 15}.
- **Grouping the 1s:**
 - $B'D'$ = corners {0,2,8,10} → **EPI** (only PI on m8, m10).
 - $A'D$ = {1,3,5,7} → PI.
 - $A'B'$ = {0,1,2,3} → PI.
 - m1 and m3 are each covered by both $A'D$ and $A'B'$ → neither is essential.
 - **Two MSPs:** $F = B'D' + A'D$ **or** $F = B'D' + A'B'$ (4 literals each).
- **Grouping the 0s:**
 - B = rows 01 and 11 = {4,5,6,7,12,13,14,15} (using X5, X7) → **EPI**.
 - AD = {9,11,13,15} → **EPI** (only one on m9, m11).
 - $A'D'$ = {0,2,4,6} → a PI (uses X0, X2) but not needed.
 - $F' = B + AD$ → **MPS:** $F = B'(A' + D')$ = $A'B' + B'D'$.
- **MSP = MPS? Not necessarily:**
 - $B'D' + A'D$ treats X5, X7 as **1** ($A'D$ covers them), but the MPS treats them as **0** (B covers them) → **different functions**.
 - $B'D' + A'B'$ happens to equal the MPS.
 - Both versions agree on every specified cell, so both are valid designs.

(c) General conclusion

- **Without don't-cares:** MSP and MPS always describe the same function (only one function fits the truth table).
- **With don't-cares:** each minimisation picks values for the Xs independently (SOP sets some Xs to 1, POS sets some to 0), so MSP and MPS **may differ as functions**. They always agree on the specified 0s and 1s. Never assume they match.

Key Logic & Takeaway

- An X is a free choice made separately by each minimisation.
- Corner groups ($B'D'$) and edge wraps are easy to miss; check them first.

Tut-4 Q2 — Gray-Coded Input with a Parity Check

(a) Completed truth table

- $V = 1$ if $G_2G_1G_0Y$ has an even number of 1s. $P = 1$ if $X \in \{2, 3, 5, 7\}$. When $V = 0$ the downstream circuit ignores P → **$P = X$ (don't-care)** on invalid rows.
- Minterm index = $8 \cdot G_2 + 4 \cdot G_1 + 2 \cdot G_0 + Y$.

X	G2G1G0	Y	m	#1s	V	P
0	000	0	0	0	1	0
0	000	1	1	1	0	X
1	001	0	2	1	0	X
1	001	1	3	2	1	0
2	011	0	6	2	1	1
2	011	1	7	3	0	X
3	010	0	4	1	0	X
3	010	1	5	2	1	1
4	110	0	12	2	1	0
4	110	1	13	3	0	X
5	111	0	14	3	0	X
5	111	1	15	4	1	1
6	101	0	10	2	1	0
6	101	1	11	3	0	X
7	100	0	8	1	0	X
7	100	1	9	2	1	1

- ⚠ EXAM TRICK: the rows are listed in Gray order, but the **minterm column** comes from the binary value of G2G1G0Y. Always compute m explicitly.

(b) V: PIs, EPIs, MSP

- $V = \Sigma m(0, 3, 5, 6, 9, 10, 12, 15)$ (even number of 1s).

G2G1 \ G0Y	00	01	11	10
00	1	0	1	0
01	0	1	0	1
11	1	0	1	0
10	0	1	0	1

- Perfect checkerboard: no two 1s are adjacent (including wrap-around).
- PIs = the 8 minterms themselves; all 8 are EPIs.**
- MSP:** $V = G_2'G_1'G_0'Y' + G_2'G_1'G_0Y + G_2'G_1G_0'Y + G_2'G_1G_0Y' + G_2G_1'G_0'Y + G_2G_1'G_0Y' + G_2G_1G_0'Y' + G_2G_1G_0Y$ (32 literals).
- Practical circuit: $V = (G_2 \oplus G_1 \oplus G_0 \oplus Y)'$ (3 XORs + 1 inverter, or 2 XOR + 1 XNOR).

(c) P with invalid rows as don't-cares

- $P = \Sigma m(5, 6, 9, 15) + d(1, 2, 4, 7, 8, 11, 13, 14)$; 0s at {0, 3, 10, 12}.

G2G1 \ G0Y	00	01	11	10
00	0	X	0	X
01	X	1	X	1
11	0	X	1	X
10	X	1	X	0

- PIs** (each is a group of 4 containing no 0):
 - $G_0'Y = \{1, 5, 9, 13\}$

- $G1Y = \{5,7,13,15\}$
- $G1G0 = \{6,7,14,15\}$
- $G2'G1 = \{4,5,6,7\}$
- $G2Y = \{9,11,13,15\}$
- $G2'G0Y' = \{2,6\}$
- $G2G1'G0' = \{8,9\}$
- **EPIs: none.** Each required 1 lies in at least two PIs (m5: $G0'Y$, $G1Y$, $G2'G1$; m6: $G1G0$, $G2'G1$, $G2'G0Y'$; m9: $G0'Y$, $G2Y$, $G2G1'G0'$; m15: $G1Y$, $G1G0$, $G2Y$). This is a **cyclic** cover.
- **Two minimal SOPs (4 literals each):**
 - $P = G0'Y + G1G0$
 - $P = G2'G1 + G2Y$
- Check $G0'Y + G1G0$ on the 0s: m0 (0000) $\rightarrow 0$ ✓; m3 (0011) $\rightarrow G0 = 1$ kills the first term, $G1 = 0$ kills the second $\rightarrow 0$ ✓; m10 (1010) $\rightarrow Y = 0$, $G1 = 0 \rightarrow 0$ ✓; m12 (1100) $\rightarrow Y = 0$, $G0 = 0 \rightarrow 0$ ✓.

(d) Force $P = 0$ on invalid rows

- $P = \Sigma m(5,6,9,15)$ with no don't-cares. These four minterms are pairwise non-adjacent (they all have even parity), so nothing merges.
- **MSP:** $P = G2'G1G0'Y + G2'G1G0Y' + G2G1'G0'Y + G2G1G0Y \rightarrow$ **16 literals**, versus **4** in (c).
- Lesson: telling the designer "the output is ignored when $V = 0$ " is worth a 4× reduction in logic.

Key Logic & Takeaway

- Parity functions are checkerboards: no K-map reduction is possible, so use XOR.
- Inputs the downstream circuit ignores are don't-cares: use them.
- No EPIs means a cyclic map with multiple equally good answers.

Tut-4 Q3 — K-map Reduction Practice

(a)

AB \ CD	00	01	11	10
00	0	0	0	X
01	0	1	1	0
11	0	0	X	0
10	X	1	1	1

- 1s $\{5, 7, 9, 10, 11\}$; Xs $\{2, 8, 15\}$.
- **PIs:** AB' $\{8,9,10,11\}$, $A'BD$ $\{5,7\}$, BCD $\{7,15\}$, ACD $\{11,15\}$, $B'CD'$ $\{2,10\}$.
- **EPIs:** AB' (only PI covering m9) and $A'BD$ (only PI covering m5).
- All 1s are covered by the EPIs.
- **MSP:** $F = AB' + A'BD$ (5 literals).

(b)

AB \ CD	00	01	11	10
00	0	X	0	X
01	X	0	1	0
11	0	0	0	0
10	1	1	1	1

- 1s {7, 8, 9, 10, 11}; Xs {1, 2, 4}.
- **PIs:** AB' {8,9,10,11}, $B'C'D$ {1,9}, $B'CD'$ {2,10}, $A'BCD$ {7}.
 - m7's neighbours m3, m5, m6, m15 are all 0, so m7 is a PI on its own.
- **EPIs:** AB' (m8, m11) and $A'BCD$ (m7).
- **MSP:** $F = AB' + A'BCD$ (6 literals).
- **EXAM TRICK:** X4 looks tempting next to m7, but they are not adjacent (0100 vs 0111 differ in 2 bits).

(c)

AB \ CD	00	01	11	10
00	1	1	1	1
01	0	X	0	X
11	X	1	0	0
10	1	0	0	1

- 1s {0, 1, 2, 3, 8, 10, 13}; Xs {5, 6, 12}.
- **PIs:** $A'B'$ {0,1,2,3}, $B'D'$ {0,2,8,10}, $BC'D$ {5,13}, ABC' {12,13}, $A'C'D$ {1,5}, $A'CD'$ {2,6}, $AC'D'$ {8,12}.
- **EPIs:** $A'B'$ (only PI on m3) and $B'D'$ (only PI on m10).
- Remaining 1: m13, covered by $BC'D$ or ABC' .
- **Two MSPs:** $F = A'B' + B'D' + BC'D$ or $F = A'B' + B'D' + ABC'$ (7 literals).

(d)

AB \ CD	00	01	11	10
00	X	0	0	X
01	0	0	1	0
11	1	0	0	0
10	X	1	1	1

- 1s {7, 9, 10, 11, 12}; Xs {0, 2, 8}.
- **PIs:** AB' {8,9,10,11}, $B'D'$ {0,2,8,10}, $AC'D'$ {8,12}, $A'BCD$ {7}.
- **EPIs:** AB' (m9, m11), $AC'D'$ (only one on m12), $A'BCD$ (m7 is isolated).
- $B'D'$ is a PI but is not needed (m10 is already covered by AB').
- **MSP:** $F = AB' + AC'D' + A'BCD$ (9 literals).

Key Logic & Takeaway

- Always look for isolated 1s first: they are their own EPIs (m7 in (b) and (d)).
- Use an X only if it enlarges a group that covers a real 1. Groups made mostly of Xs (like $B'D'$ in (d)) are PIs, but you often do not need them.
- When the remaining 1s can be covered in two ways, give both MSPs.

TUTORIAL 5 — Decoders, Multiplexers and Their Applications

Tut-5 Q1 — Decoders as Minterm and Maxterm Generators

Background in 3 bullets

- Active-high n -to- 2^n decoder: output $D_k = m_k$ (minterm k) → OR the 1-rows to get F .
- Active-low decoder: output $D_k' = m_k' = M_k$ (maxterm k) → AND the 0-rows (POS) **or** NAND the 1-rows (SOP).
- A single required minterm/maxterm needs **no gate at all**.

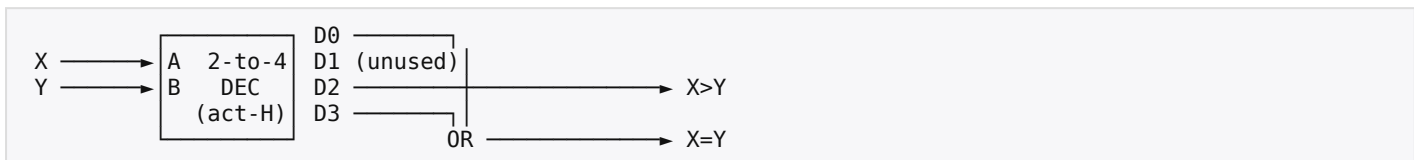
(a) Active-high decoder

(a)1 — 1-bit comparator with one 2-to-4 decoder + one OR

- Decoder inputs X (MSB), $Y \rightarrow D_0 = X'Y'$, $D_1 = X'Y$, $D_2 = XY'$, $D_3 = XY$.

X	Y	$X > Y$	$X = Y$
0	0	0	1
0	1	0	0
1	0	1	0
1	1	0	1

- $X > Y = m_2 = D_2 \rightarrow$ **wire only, no gate**.
- $X = Y = m_0 + m_3 = D_0 + D_3 \rightarrow$ **the one OR gate**.



(a)2 — $H(A,B,C) = A'C + AB'C' + ABC$ with a 3-to-8 decoder

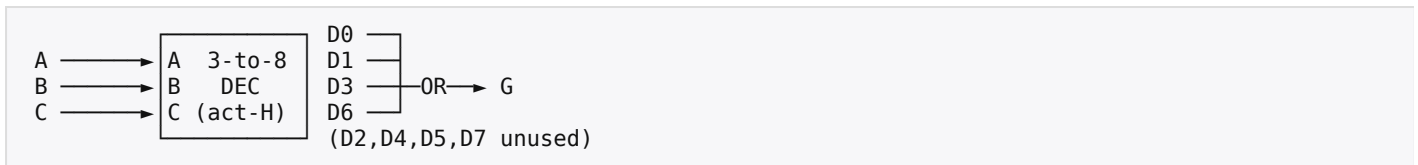
- Expand to minterms:
 - $A'C = A'B'C + A'BC = m_1 + m_3$
 - $AB'C' = m_4$
 - $ABC = m_7$
- $H = \Sigma m(1,3,4,7) \rightarrow$ **4-input OR of D1, D3, D4, D7**.

(a)3 — $F(X,Y) = \Sigma m(0,2,3)$ with a 2-to-4 decoder

- $F = D_0 + D_2 + D_3$ (3-input OR).
- Cheaper: F has a single 0 (m_1), so $F = (m_1)' = D_1' \rightarrow$ **one inverter**, or a NOR with only D_1 as input.
- ⚠ EXAM TRICK: always compare #1s and #0s and use the smaller side.

(a)4 — $G(A,B,C) = \Sigma m(0,1,3,6)$ with a 3-to-8 decoder

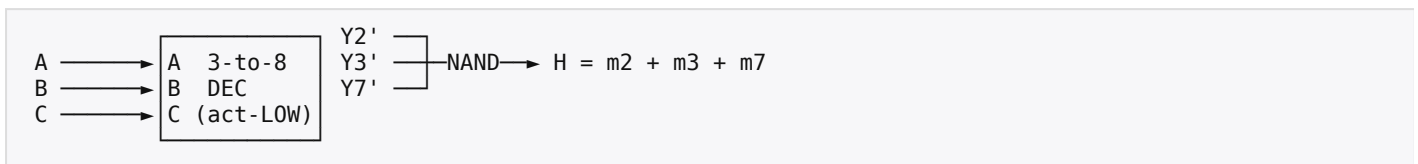
- $G = D_0 + D_1 + D_3 + D_6$ (4-input OR). Equally, NOR of D_2, D_4, D_5, D_7 (also 4 inputs).



(b) Active-low decoder (outputs are maxterms Mk)

(b)1 — $H(A,B,C) = B \cdot (A+B+C') \cdot (A'+B'+C)$ with a 3-to-8 decoder

- Turn each factor into maxterms:
 - $B = 0$ whenever $B = 0 \rightarrow$ rows 000, 001, 100, 101 $\rightarrow M_0, M_1, M_4, M_5$.
 - $A + B + C' = 0$ at 001 $\rightarrow M_1$ (already included).
 - $A' + B' + C = 0$ at 110 $\rightarrow M_6$.
- $H = \prod M(0, 1, 4, 5, 6) = \sum m(2, 3, 7)$.
- Two options:
 - AND of active-low outputs 0, 1, 4, 5, 6 $\rightarrow$ **5-input AND**.
 - NAND of active-low outputs 2, 3, 7 $\rightarrow$ 3-input NAND $\checkmark$ (cheaper).**
- Why the NAND works: $\text{NAND}(M_2, M_3, M_7) = M_2' + M_3' + M_7' = m_2 + m_3 + m_7$.



(b)2 — $F(X,Y) = \prod M(1)$ with a 2-to-4 decoder

- A single maxterm: $F = M_1 = X + Y' =$ active-low output $Y1'$ itself. **No gate: wire $Y1'$ to F .**

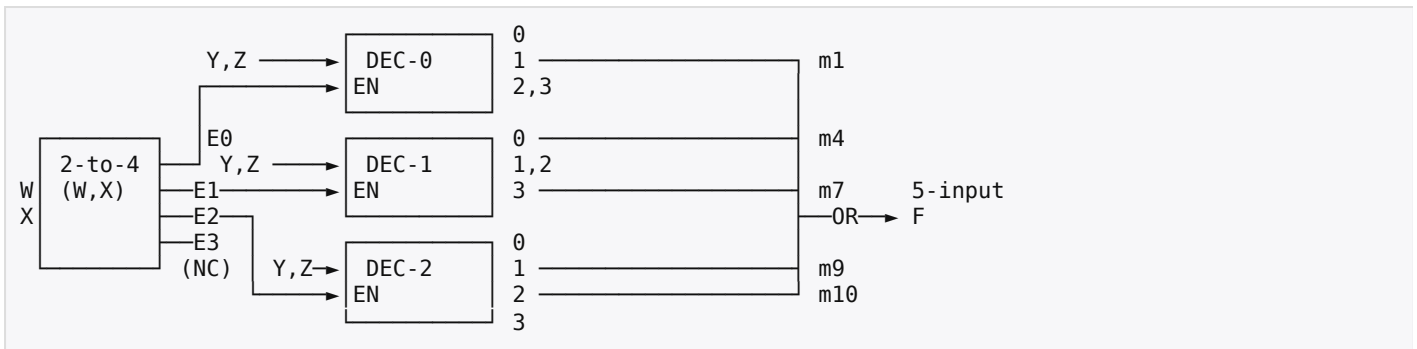
(b)3 — $G(A,B,C) = \prod M(2,4,5,7)$ with a 3-to-8 decoder

- 4-input AND of active-low outputs 2, 4, 5, 7.**
- Equivalent: $G = \sum m(0, 1, 3, 6)$ (the same function as (a)4) $\rightarrow$ 4-input NAND of outputs 0, 1, 3, 6. Same cost.

(c) Hierarchical: $F(W,X,Y,Z) = \sum m(1,4,7,9,10)$ using only 2-to-4 decoders + one OR

- Build a 4-to-16 decoder as a tree: one 2-to-4 on (W, X) drives the **enables** of 2-to-4 decoders on (Y, Z).
- The decoder for $WX = 11$ (minterms 12-15) is not needed $\rightarrow$ **4 decoders + 1 OR**.

Minterm	WXYZ	Enabled by	Output used
1	00 01	E0 (W'X')	DEC-0 output 1
4	01 00	E1 (W'X)	DEC-1 output 0
7	01 11	E1	DEC-1 output 3
9	10 01	E2 (WX')	DEC-2 output 1
10	10 10	E2	DEC-2 output 2



- ⚠ EXAM TRICK: the second-level decoders must have an enable input. The MSB pair (W, X) goes on the first level; if you swap it with (Y, Z), the minterm numbering comes out permuted.

Key Logic & Takeaway

- Active-high = minterms → OR (1-rows) / NOR (0-rows). Active-low = maxterms → AND (0-rows) / NAND (1-rows).
- Count the 1s and 0s and pick the gate with fewer inputs; a lone minterm or maxterm is just a wire (or inverter).

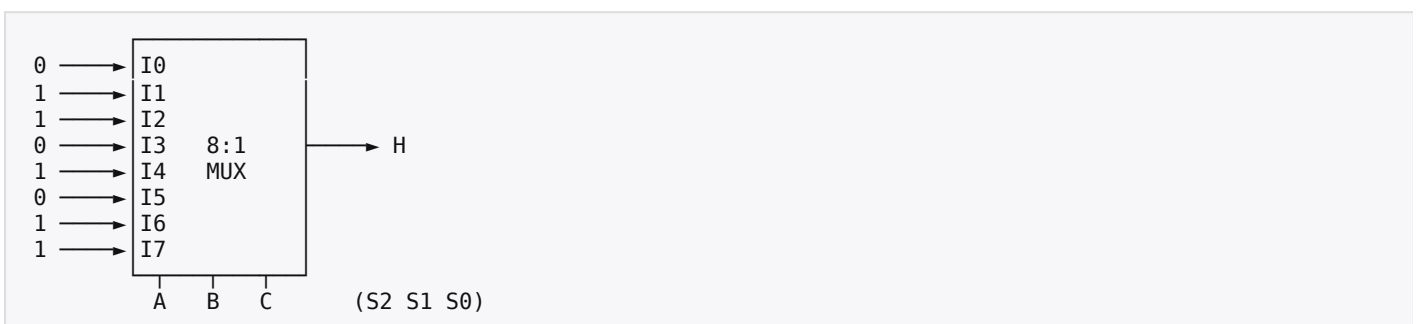
Tut-5 Q2 — Implementing Expressions with Multiplexers

(a) 3-variable functions on an 8-to-1 mux (all variables on the selects)

- Rule: $I_k = F$ at row k .

Function	Minterms	I0	I1	I2	I3	I4	I5	I6	I7
1. $H = (A+B+C)(A+B'+C')(A'+B+C') = \Pi M(0,3,5)$	$\Sigma m(1,2,4,6,7)$	0	1	1	0	1	0	1	1
2. $F = \Sigma m(0,2,5,7)$	0,2,5,7	1	0	1	0	0	1	0	1
3. $G = \Sigma m(1,3,4,5,6)$	1,3,4,5,6	0	1	0	1	1	1	1	0

- For H: $(A+B+C) = 0$ at 000 → M0; $(A+B'+C') = 0$ at 011 → M3; $(A'+B+C') = 0$ at 101 → M5.



(b) 4-variable functions on an 8-to-1 mux (implementation tables)

(b)1 — $G(A,B,C,D) = \Sigma m(2,4,6,12,13,15)$, selects B, C, D; A on the data lines

- ⚠ EXAM TRICK: here the **MSB A** is the data variable, not D. Select value $k = BCD$; the two minterms in column k are k ($A = 0$) and $k + 8$ ($A = 1$).

BCD = k	000	001	010	011	100	101	110	111
A = 0 (m = k)	0	1	2	3	4	5	6	7
A = 1 (m = k+8)	8	9	10	11	12	13	14	15
lk	0	0	A'	0	1	A	A'	A

(b)2 — $H(A,B,C,D) = \Sigma m(1,4,11,14) + d(2,7)$, selects A, B, C; D on the data lines

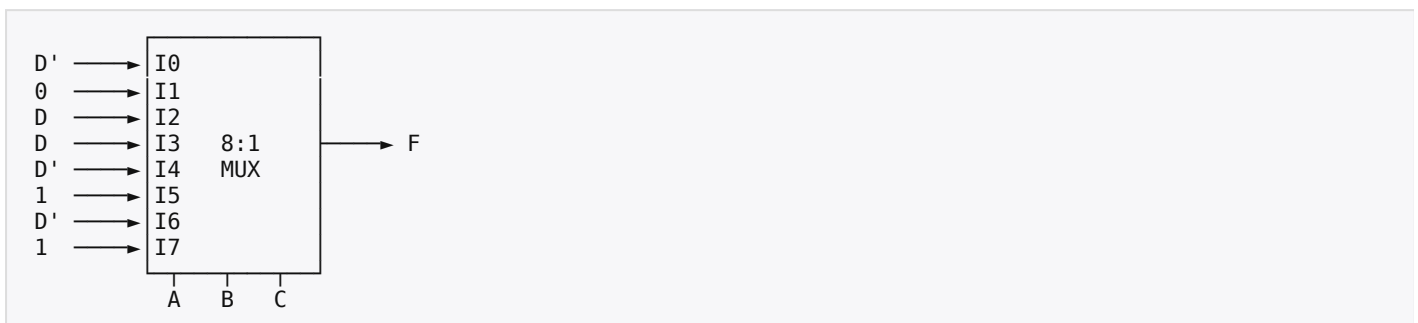
- Select k = ABC covers minterms 2k (D = 0) and 2k + 1 (D = 1).

ABC = k	000	001	010	011	100	101	110	111
D = 0	0	X2	4	6	8	10	12	14
D = 1	1	3	5	X7	9	11	13	15
lk	D	0	D'	0	0	D	0	D'

- Xs: column 001 has X2 and 0 → choose X2 = 0 → **0** (a constant is simplest). Column 011 has 0 and X7 → **0**.
- The Xs could also give **D'** and **D**, but constants are cheaper.

(b)3 — $F(A,B,C,D) = \Sigma m(0,5,7,8,10,11,12,14,15)$, selects A, B, C; D on the data lines

ABC = k	000	001	010	011	100	101	110	111
D = 0	0	2	4	6	8	10	12	14
D = 1	1	3	5	7	9	11	13	15
lk	D'	0	D	D	D'	1	D'	1

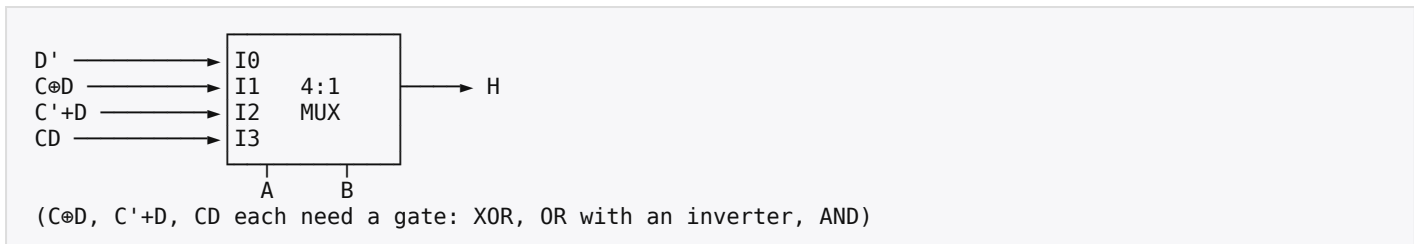


(c) $H(A,B,C,D) = \Sigma m(0,2,5,6,8,9,11,15)$ on a 4-to-1 mux (selects A, B)

- Each data input is a function of C, D: read the K-map row for that AB value (cells in order CD = 00, 01, 11, 10).

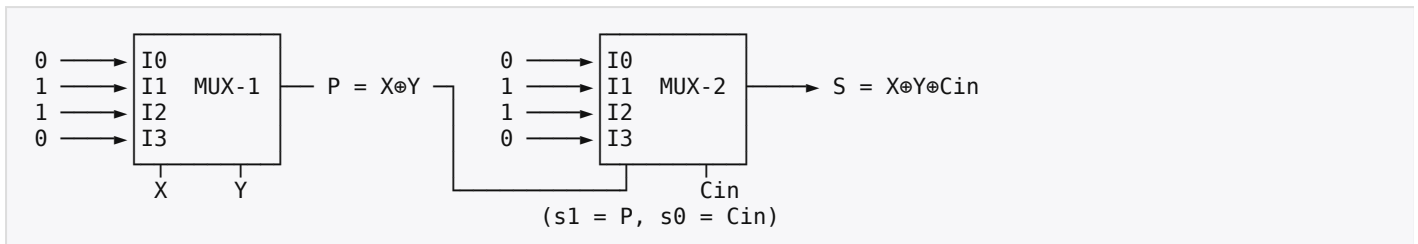
AB \ CD	00	01	11	10	lk
00 (I0)	1 (m0)	0	0	1 (m2)	D'
01 (I1)	0	1 (m5)	0	1 (m6)	$C \oplus D = C'D + CD'$
11 (I3)	0	0	1 (m15)	0	CD
10 (I2)	1 (m8)	1 (m9)	1 (m11)	0	$C' + D$

- ⚠ EXAM TRICK: the K-map lists row 11 **before** row 10, but the mux input for AB = 10 is **I2** and for AB = 11 is **I3**.



(d) Full-adder sum with only two 4-to-1 muxes

- $S = X \oplus Y \oplus Cin = (X \oplus Y) \oplus Cin$.
- A 4:1 mux with data 0, 1, 1, 0 is a 2-input XOR of its select lines ($Y = S1'S0 + S1S0'$).
- **MUX-1:** selects (X, Y), data 0,1,1,0 $\rightarrow P = X \oplus Y$.
- **MUX-2:** selects (P, Cin), data 0,1,1,0 $\rightarrow S = P \oplus Cin$.



- Alternative (if complements are free): one mux with selects (Y, Cin) and data X, X', X', X already gives S. Two muxes are needed only when X' is not available; MUX-1 can then be used as an inverter.

Key Logic & Takeaway

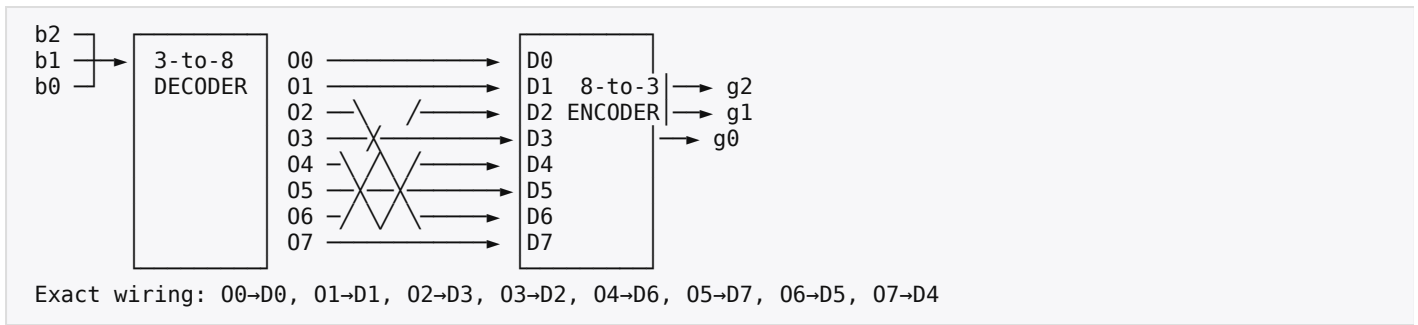
- n vars on selects $\rightarrow$ constants. $n-1$ on selects $\rightarrow \{0, 1, v, v'\}$. $n-2$ on selects $\rightarrow$ small functions (K-map each row).
- Watch which variable is on the data line (MSB in (b)1!) and the AB = 10 / 11 swap between K-map rows and mux inputs.
- A 4:1 mux with data 0,1,1,0 is an XOR gate.

Tut-5 Q3 — Application Questions

Q3.1 — 3-bit Binary $\rightarrow$ Gray with one 3-to-8 decoder + one 8-to-3 encoder

- The decoder turns binary $b2b1b0$ into one active line k ; wiring line k to encoder input $g(k)$ makes the encoder output the Gray code.

Binary k	b2b1b0	Gray	Encoder input
0	000	000	D0
1	001	001	D1
2	010	011	D3
3	011	010	D2
4	100	110	D6
5	101	111	D7
6	110	101	D5
7	111	100	D4

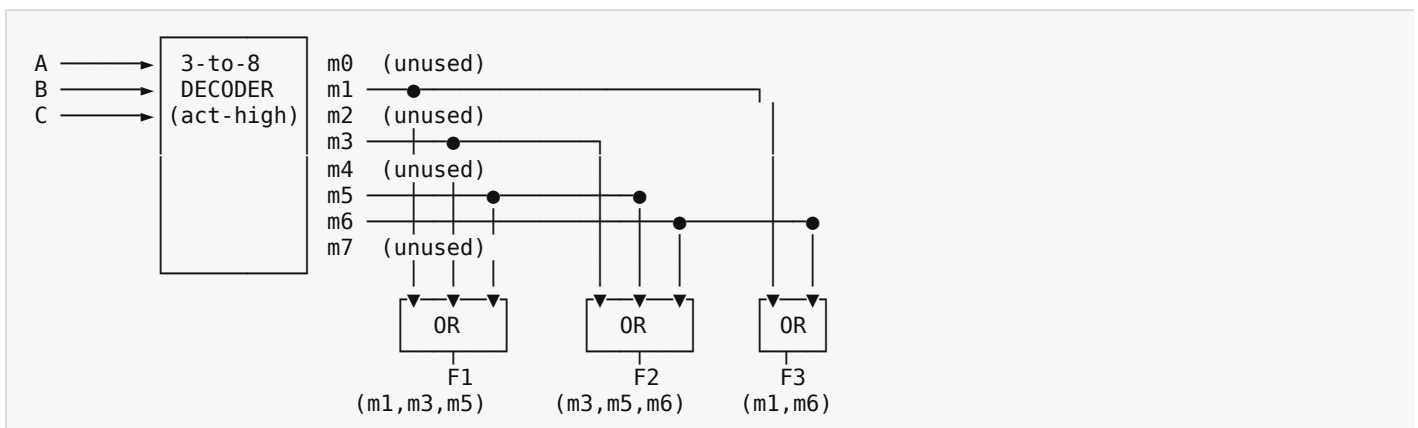


- Check: input 110 (6) → 06 → D5 → output 101 ✓ ($\text{gray}(6) = 6 \oplus 3 = 101$).

Q3.2 — Sharing one decoder across three functions

- $F1 = \Sigma m(1,3,5)$, $F2 = \Sigma m(3,5,6)$, $F3 = \Sigma m(1,6)$.

(a) One 3-to-8 decoder + one OR per function



Decoder line	→ F1	→ F2	→ F3
m1	✓		✓
m3	✓	✓	
m5	✓	✓	
m6		✓	✓

(b) AND gates used vs wasted

- Minterms used by at least one function: {1, 3, 5, 6} → **4 used**.
- Never used: {0, 2, 4, 7} → **4 wasted** (half the decoder).

(c) A custom AND array (the PLA idea)

- One AND gate per distinct product term actually needed: m1, m3, m5, m6 → **4 AND gates** (vs 8).
- Fan-out to the OR gates:

AND gate	Feeds
$m1 = A'B'C$	F1, F3
$m3 = A'BC$	F1, F2
$m5 = AB'C$	F1, F2
$m6 = ABC'$	F2, F3

- **Every one of the 4 AND gates feeds more than one OR gate.** This sharing is exactly how a **PLA** saves area. Minimising each function separately (e.g. $F1 = A'C + B'C$) would create new terms nobody else can share and raise the count to 6.

Q3.3 — Driving a multiplexed 4-digit display

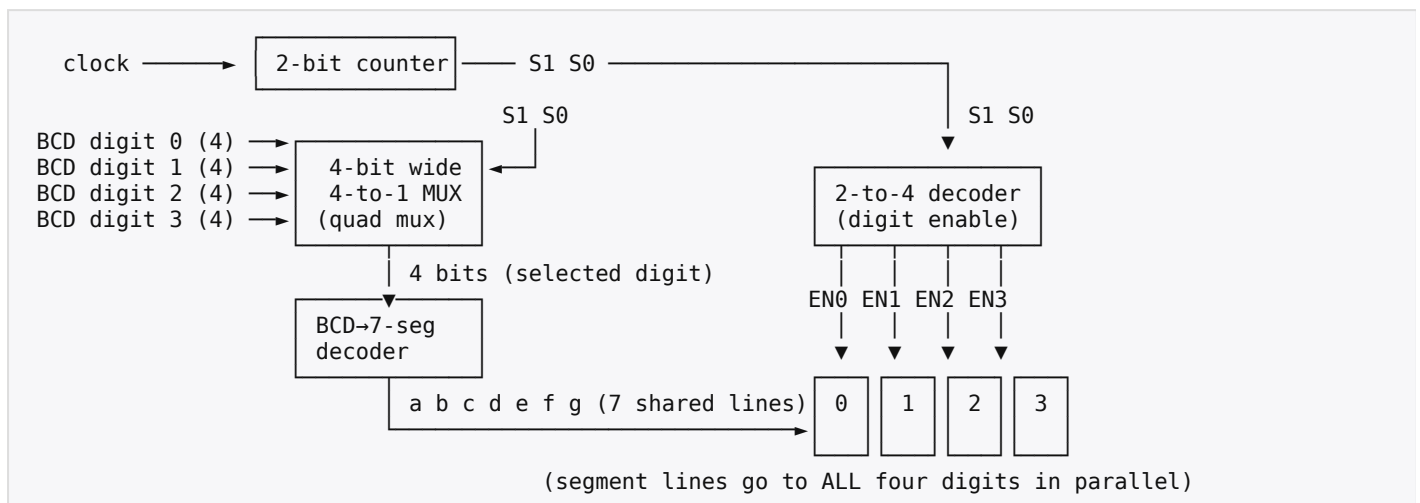
(a) Wire count

- Dedicated drivers: 4 digits × 7 segments = **28** segment wires (plus 4 decoders).
- Shared: **7** segment lines + **4** digit-enable (common) lines = **11** wires.
 - With the 2-to-4 decoder mounted at the display end, only 2 select lines need to run → $7 + 2 = 9$.

(b) Digit-enable decoder

- A 2-bit counter steps $S1S0 = 00 \rightarrow 01 \rightarrow 10 \rightarrow 11 \rightarrow 00 \dots$ fast ($\geq \sim 60$ Hz per digit, so the eye sees steady digits).
- A 2-to-4 decoder (= 1-to-4 demux with data = 1) turns $S1S0$ into exactly one active digit-enable line:
 - $EN0 = S1'S0'$, $EN1 = S1'S0$, $EN2 = S1S0'$, $EN3 = S1S0$.
- Common-cathode digits need their common line pulled **low** to light → use an **active-low** decoder (or invert the outputs).

(c) Complete system



(d) Why the mux and the decoder must see the same 2-bit value

- At each instant the segment lines carry the pattern of the digit selected by the **mux**, and it lights on whichever digit the **decoder** enables.
- If they differ (say the mux selects digit 2's value while the decoder enables digit 1), digit 1 shows digit 2's number.
- Even a slight skew at each switching edge briefly shows the next or previous digit's pattern on the wrong position → **ghosting**: faint wrong segments on neighbouring digits.
- Hence both blocks are driven by the **same wires** from the same counter. Real designs also blank the display briefly during the switch.

Key Logic & Takeaway

- Decoder → permuted wiring → encoder = any one-to-one code converter.

- A decoder wastes AND gates for unused minterms; a shared AND array (PLA) builds only the terms needed and shares them.
- Time-multiplexing = mux (data) + demux/decoder (destination) driven by the same select lines.

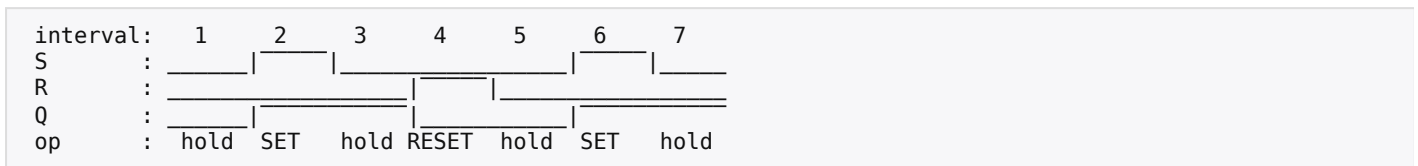
TUTORIAL 6 — Latches, Flip-Flops and Sequential Circuit Analysis

Tut-6 Q1 — Complete the Timing Diagrams

(a) Basic SR latch (NOR-based), Q starts at 0

- NOR SR latch rules: $SR = 00$ hold, 10 set ($Q = 1$), 01 reset ($Q = 0$), 11 forbidden (both outputs 0).
- From the figure: S is high in intervals 2 and 6; R is high in interval 4.

Interval	1	2	3	4	5	6	7
S	0	1	0	0	0	1	0
R	0	0	0	1	0	0	0
Q	0	1	1	0	0	1	1
Operation	hold	set	hold	reset	hold	set	hold

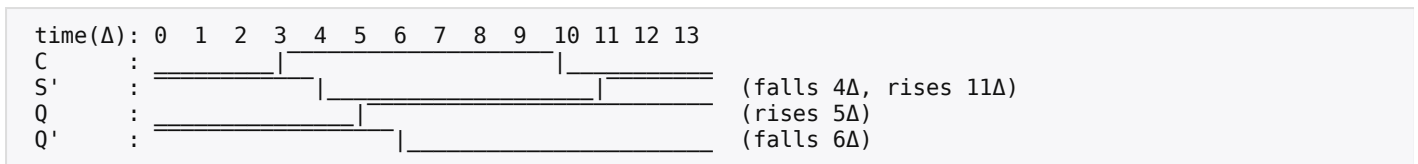


- ⚠ EXAM TRICK: when S returns to 0 (interval 3) Q does **not** fall: the latch remembers. Only R resets it.

(b) Positive-edge D flip-flop with asynchronous Reset, Q starts at 0

- Reading the figure (one time unit = half a clock period):
 - Rising clock edges at **t1, t3, t5, t7, t9**.
 - $D = 1$ until $t4$, $D = 0$ from $t4$ to $t6$, $D = 1$ after $t6$.
 - Reset is high from **t4.5 to t5.5**.

Time	Event (cause → effect)
3Δ	C rises
4Δ	NAND1: $S \cdot C = 1 \rightarrow S' \text{ falls to } 0$ (1 gate delay)
5Δ	NAND3: $S' = 0 \rightarrow Q \text{ rises to } 1 \leftarrow Q \text{ begins to change}$
6Δ	NAND4: $R' = 1, Q = 1 \rightarrow Q' \text{ falls to } 0 \leftarrow \text{circuit fully stable}$
7Δ	NAND3 now sees $S' = 0, Q' = 0 \rightarrow Q \text{ stays } 1$ (no further change)
10Δ	C falls
11Δ	S' returns to 1
12Δ	NAND3 sees $S' = 1, Q' = 0 \rightarrow Q = 1$: no change



- **Q first begins to change at 5Δ** (2Δ after C rises). **Fully stable at 6Δ** (3Δ after C rises), once the Q' feedback has arrived.
- **When C falls at 10Δ, Q does not change.** S' and R' both return to 1, which is the "hold" input for the cross-coupled NAND pair. Since Q' = 0 is already fed back, NAND3 keeps Q = 1. The latch stores the value.
- (Verified with a gate-level delay simulation.)

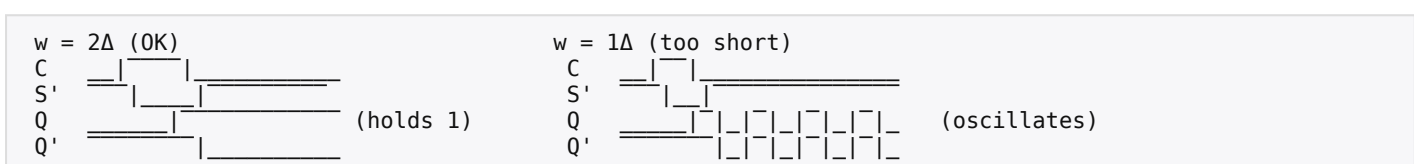
Key Logic & Takeaway

- Count gate levels along the path: C → input NAND (Δ) → output NAND (Δ) → other output NAND (Δ).
- A latch is "settled" only when **both** outputs have changed and the feedback loop agrees.

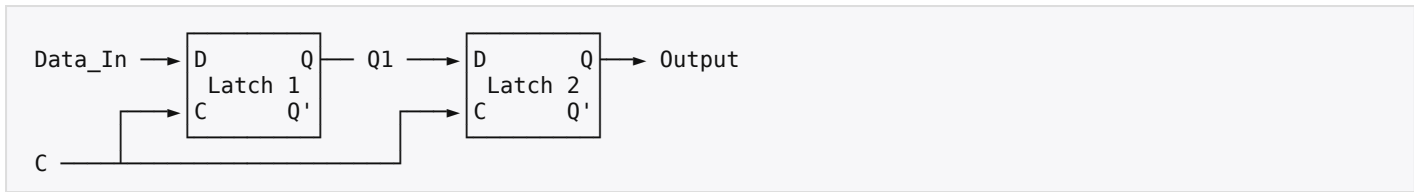
Tut-6 Q3 — Using Latches to Manipulate Signals

(a) Minimum C pulse width w

- When C goes high, S' falls Δ later; Q rises Δ after that; Q' falls Δ after that.
- The latch holds by itself only once **Q' = 0** has fed back into NAND3. Until then, Q is 1 only because S' is still 0.
- S' stays low until Δ after C falls (i.e. until w + Δ). It must still be low when Q' arrives at 3Δ:
 - $w + \Delta \geq 3\Delta \rightarrow w_{\min} = 2\Delta$ (the time for the change to travel once around the feedback loop).
- **If w < 2Δ:** S' returns to 1 before Q' = 0 has arrived, so NAND3 sees (S' = 1, Q' = 1) and Q falls back to 0. Meanwhile the Q' = 0 pulse arrives and pushes Q back up. In the equal-delay model, Q and Q' then **oscillate** indefinitely (simulated: w = 0.5Δ, 1Δ, 1.5Δ all oscillate; w = 2Δ latches cleanly). In real hardware this is **metastability**: Q may hover between levels and finally settle to 0 (old value) or 1 at random.
- So a too-short pulse means Q does not reliably hold the new value.



(b) Two latches in series sharing C (intended 1-bit shift per pulse)



- **Yes, there is a maximum safe width.**
- While $C = 1$, **both** latches are transparent. Latch 1's Q1 becomes Data_In one propagation delay t_{pd} ($\approx 2-3\Delta$ for these latches) after C rises.
- If C is still high when the new Q1 arrives at Latch 2's input, Latch 2 (also transparent) passes it straight through $\rightarrow$ after the pulse **both latches hold Data_In**, and the old value of Latch 1 is lost. This is **race-through**: the data moved two stages in one pulse.
- **Condition:** $w < t_{pd}(\text{Latch 1}, C \rightarrow Q)$.
- **The real problem:** the minimum width from (a) is $\approx 2\Delta$ and the maximum is also $\approx 2\Delta$ (one latch delay). With identical latches the valid window is about zero width, so the circuit cannot be made reliable over temperature and process variation.

(c) How edge-triggered flip-flops remove the problem

- An edge-triggered (e.g. master-slave) flip-flop samples D only in a tiny window around the clock edge (setup + hold time) and changes Q only **after** the edge.
- **Internally:** for a positive-edge FF the master latch is transparent while $C = 0$ and the slave while $C = 1$. They are **never transparent at the same time**, so data cannot race through.
- In a chain, FF2 samples Q1 at the same edge that FF1 samples Data_In. Q1 only changes t_{pd} after the edge, which is longer than FF2's hold time, so FF2 captures the **old** Q1. Exactly one shift per edge.
- Pulse width no longer matters (beyond a minimum high/low time); there is **no maximum**. Timing reduces to setup/hold checks at one instant.

Key Logic & Takeaway

- Latches have both a minimum pulse width (loop must close: 2Δ) and a maximum (no race-through: $< t_{pd}$). For identical latches these coincide $\rightarrow$ unreliable.
- Edge triggering = master + slave latches on opposite clock phases $\rightarrow$ only the edge matters.

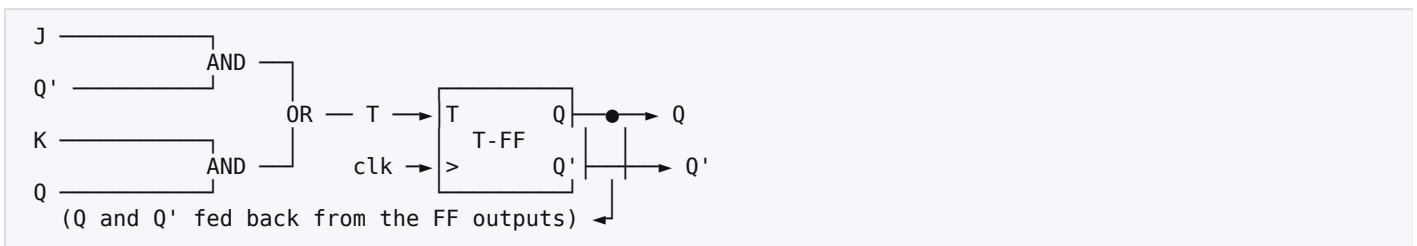
Tut-6 Q4 — Design a JK Flip-Flop Using a T Flip-Flop

- A T flip-flop toggles when $T = 1$. So **$T = 1$ exactly when the JK flip-flop must change state:** $T = Q \oplus Q+$.

J	K	Q	Q+ (JK behaviour)	T = Q ⊕ Q+
0	0	0	0 (hold)	0
0	0	1	1 (hold)	0
0	1	0	0 (reset)	0
0	1	1	0 (reset)	1
1	0	0	1 (set)	1
1	0	1	1 (set)	0
1	1	0	1 (toggle)	1
1	1	1	0 (toggle)	1

J \ K Q	00	01	11	10
0	0	0	1	0
1	1	0	1	1

- Groups: $JQ' = \{m4, m6\}$ ($J = 1, Q = 0$ column pair 00 and 10 wrap) and $KQ = \{m3, m7\}$. Both are EPIs.
- $T = JQ' + KQ$.
- Sanity check: $Q = 0 \rightarrow T = J$ (set or toggle from 0 needs a toggle); $Q = 1 \rightarrow T = K$ (reset or toggle from 1 needs a toggle).



Key Logic & Takeaway

- Converting flip-flop X into Y: write Y's characteristic table, then use **X's excitation table** to find X's inputs, then a K-map.
- For a T target: $T = Q \oplus Q+$.

Tut-6 Q5 — A New Flip-Flop Type (A, B): D-based vs JK-based

A	B	Q+
0	0	0 (reset)
0	1	Q' (toggle)
1	0	Q' (toggle)
1	1	1 (set)

Expanded table with both implementations

A	B	Q	Q+	D	J	K
0	0	0	0	0	0	X
0	0	1	0	0	X	1
0	1	0	1	1	1	X
0	1	1	0	0	X	1
1	0	0	1	1	1	X
1	0	1	0	0	X	1
1	1	0	1	1	1	X
1	1	1	1	1	X	0

(a) Using a D flip-flop: $D = Q+$

- $D = \Sigma m(2,4,6,7)$ over (A, B, Q):

A \ B Q	00	01	11	10
0	0	0	0	1
1	1	0	1	1

- Pls (all essential): BQ' {2,6}, AQ' {4,6}, AB {6,7}.
- $D = AQ' + BQ' + AB \rightarrow$ **6 literals** (3 two-input ANDs + a 3-input OR).

(b) Using a JK flip-flop

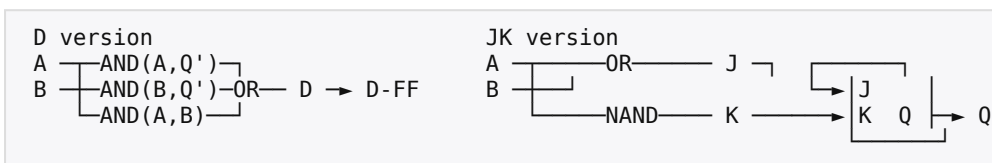
- **J** (only rows with $Q = 0$ matter; $Q = 1$ rows are X):

A \ B Q	00	01	11	10
0	0	X	X	1
1	1	X	X	1

- $J = A + B$ (2 literals).
- **K** (only rows with $Q = 1$ matter; $Q = 0$ rows are X):

A \ B Q	00	01	11	10
0	X	1	1	X
1	X	1	0	X

- $K = 0$ only at $A = B = 1 \rightarrow K = A' + B' = (AB)'$ (2 literals; a single NAND gate).
- **Total: 4 literals** (one OR + one NAND).



Comparison

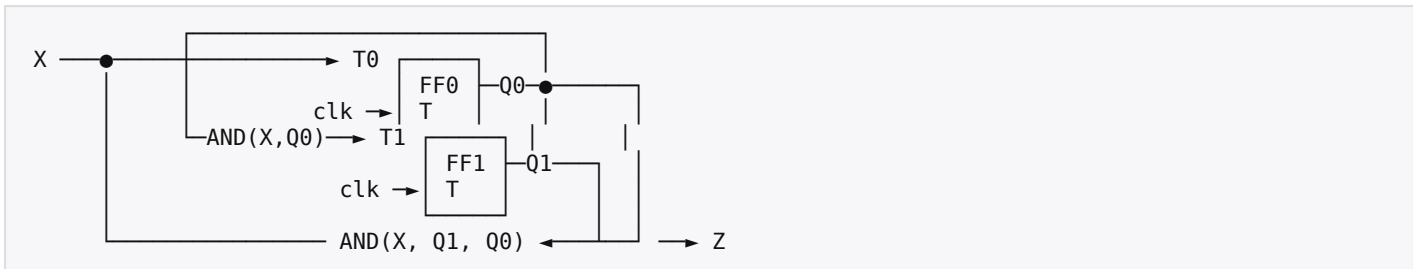
- **D: 6 literals vs JK: 4 literals $\rightarrow$ the JK version is cheaper.**
- **Why:** every row of the JK excitation table has one don't-care (when $Q = 0$, K is free; when $Q = 1$, J is free). Half the cells in each of the J and K maps are X, giving large groups. The D excitation table has **no** don't-cares (D must equal $Q+$ exactly), so its map is fully specified.

Key Logic & Takeaway

- JK maps: half X's → small equations. D maps: no X's → simple to derive, usually more logic.
- Read J from the $Q = 0$ half and K from the $Q = 1$ half.

Tut-6 Q6 — Analyse This Sequential Circuit

- Given: two T flip-flops (Q_1 Q_0), input X , output Z :
 - $T_1 = X \cdot Q_0$
 - $T_0 = X$
 - $Z = X \cdot Q_1 \cdot Q_0$ (depends on the input → **Mealy** output)



(a) State table

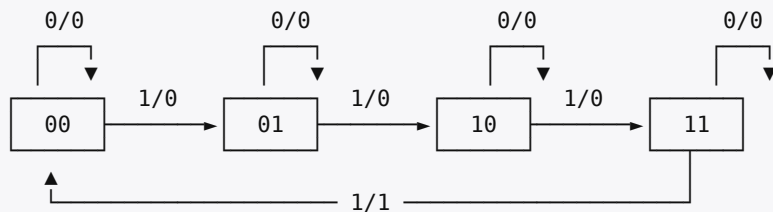
- Procedure: compute T_1 , T_0 from the present state and input; apply $Q_+ = Q \oplus T$ to each flip-flop.

Q_1 Q_0	X	$T_1 = XQ_0$	$T_0 = X$	Q_1+ Q_0+	$Z = XQ_1Q_0$
0 0	0	0	0	0 0	0
0 0	1	0	1	0 1	0
0 1	0	0	0	0 1	0
0 1	1	1	1	1 0	0
1 0	0	0	0	1 0	0
1 0	1	0	1	1 1	0
1 1	0	0	0	1 1	0
1 1	1	1	1	0 0	1

- Compact form (next state / output):

Present	$X = 0$	$X = 1$
00	00 / 0	01 / 0
01	01 / 0	10 / 0
10	10 / 0	11 / 0
11	11 / 0	00 / 1

(b) State diagram (Mealy: arcs labelled X/Z)



(c) Plain-language description

- A **2-bit (mod-4) up-counter with enable X**: it holds when $X = 0$ and counts up by one on each clock edge where $X = 1$.
- **Z = 1** for the cycle in which the counter is at 3 and $X = 1$ (it is about to wrap $3 \rightarrow 0$). So Z marks every **4th** cycle in which $X = 1$, acting as a carry-out / divide-by-4 of the X events.

Key Logic & Takeaway

- Analysis recipe: FF input equations $\rightarrow$ FF input values per row $\rightarrow$ next state via the characteristic equation ($Q^+ = T \oplus Q$) $\rightarrow$ state table $\rightarrow$ diagram $\rightarrow$ describe.
- An output that depends on X is Mealy (shown on the arcs); one that depends only on the state is Moore (shown inside the circles).
- $T_0 = X$, $T_1 = XQ_0$ is the standard synchronous binary counter pattern ($T_i = EN \cdot Q_0 \dots Q_{i-1}$, Lec-22).

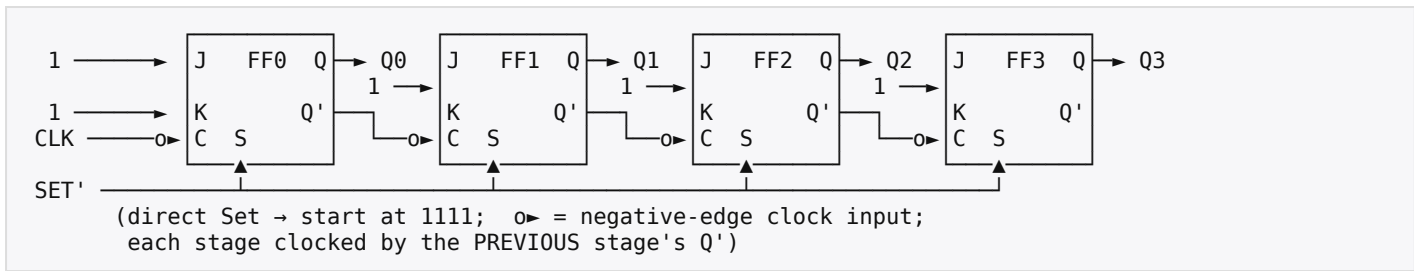
TUTORIAL 7 — Sequential Design and Counters (register questions excluded)

Tut-7 **Q4** (bottle line: a 5-stage shift-register delay) and **Q5** (serial adder/subtractor built on shift registers) are **register** questions and are skipped as you asked. Q1-Q3 are below.

Tut-7 Q1 — Binary Counters: Down, then Up-Down

(a) 4-bit ripple (asynchronous) DOWN counter

- **How the up counter works:** every FF has $J = K = 1$ (always toggles). FF0 is clocked by CLK; FF(i) is clocked by **$Q(i-1)$** on its **negative** edge. Bit i must toggle exactly when bit (i-1) goes **1 $\rightarrow$ 0** (a carry: $0111 \rightarrow 1000$).
- **What changes for counting down:** bit i must toggle exactly when bit (i-1) goes **0 $\rightarrow$ 1** (a borrow: $1000 \rightarrow 0111$ means Q_0 $0 \rightarrow 1$, then Q_1 $0 \rightarrow 1$, then Q_2 $0 \rightarrow 1$, then Q_3 $1 \rightarrow 0$).
 - A $0 \rightarrow 1$ change on $Q(i-1)$ is a $1 \rightarrow 0$ change on **$Q'(i-1)$** .
 - **So clock each stage from the previous stage's Q' instead of Q** (keeping negative-edge FFs). Equivalently, keep Q but use positive-edge-triggered FFs.
- Initialise with the **direct Set (preset)** inputs $\rightarrow 1111$.



- **Trace** (first few clocks):

CLK edge	Q0	Q0' edge	Q1	Q1' edge	Q2	Q3	Count
(preset)	1	—	1	—	1	1	15
1	1→0	0→1 (rising, ignored)	1	—	1	1	14
2	0→1	1→0 (falling) → FF1 toggles	1→0	0→1 (ignored)	1	1	13
3	1→0	ignored	0	—	1	1	12
4	0→1	falling → FF1 toggles	0→1	falling → FF2 toggles	1→0	1	11

- Continues 10, 9, ..., 1, 0, then 0000 → 1111 (every stage borrows) and repeats.
- ⚠ EXAM TRICK: summary table for ripple counters:

FF triggering	Clock next stage from Q	Clock next stage from Q'
Negative edge	UP counter	DOWN counter
Positive edge	DOWN counter	UP counter

(b) 3-bit synchronous UP/DOWN counter (UD = 0 up, UD = 1 down)

Using D flip-flops (D = next state) — K-maps use rows UD Q2, columns Q1 Q0:

- **D0**: the LSB always toggles.

UD Q2 \ Q1 Q0	00	01	11	10
00	1	0	0	1
01	1	0	0	1
11	1	0	0	1
10	1	0	0	1

- **D0 = Q0'** (1 literal).
- **D1**:

UD Q2 \ Q1 Q0	00	01	11	10
00	0	1	0	1
01	0	1	0	1
11	1	0	1	0
10	1	0	1	0

- Checkerboard in UD, Q1, Q0 → no merging beyond Q2: **D1 = UD'Q1'Q0 + UD'Q1Q0' + UDQ1'Q0' + UDQ1Q0** (12 literals) = $Q1 \oplus Q0 \oplus UD$.
- **D2**: $D2 = \Sigma m(3,4,5,6,8,13,14,15)$

UD Q2 \ Q1 Q0	00	01	11	10
00	0	0	1	0
01	1	1	0	1
11	0	1	1	1
10	1	0	0	0

- EPIs: $UD'Q2'Q1Q0$ (m3, isolated), $UDQ2'Q1'Q0'$ (m8, isolated).
- $D2 = UD'Q2'Q1Q0 + UDQ2'Q1'Q0' + Q2Q1'Q0 + UD'Q2Q0' + UDQ2Q1$ (17 literals; an equally minimal alternative is $\dots + Q2Q1Q0' + UD'Q2Q1' + UDQ2Q0$).
- **D total = 1 + 12 + 17 = 30 literals.**

Using T flip-flops (T = 1 when the bit changes)

- **T0 = 1** (always toggles) → 0 literals.
- **T1:** up: Q1 toggles when Q0 = 1; down: Q1 toggles when Q0 = 0.

UD Q2 \ Q1 Q0	00	01	11	10
00	0	1	1	0
01	0	1	1	0
11	1	0	0	1
10	1	0	0	1

- $T1 = UD'Q0 + UDQ0' = UD \oplus Q0$ (4 literals).
- **T2:** up: toggles when Q1Q0 = 11; down: toggles when Q1Q0 = 00.

UD Q2 \ Q1 Q0	00	01	11	10
00	0	0	1	0
01	0	0	1	0
11	1	0	0	0
10	1	0	0	0

- $T2 = UD'Q1Q0 + UDQ1'Q0'$ (6 literals).
- **T total = 0 + 4 + 6 = 10 literals.**

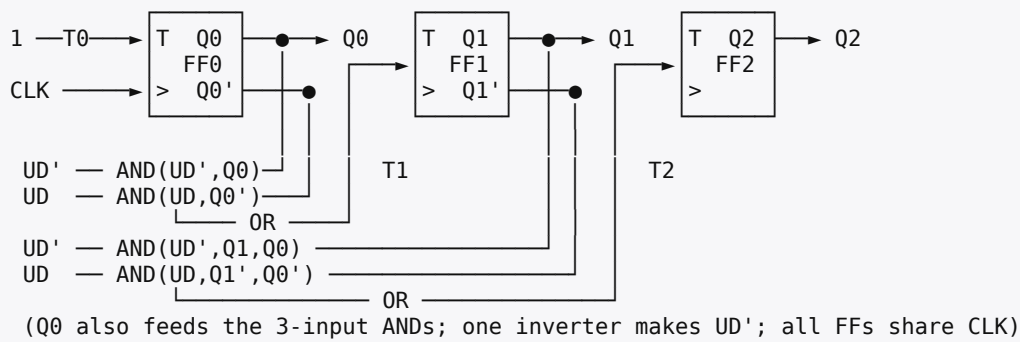
Comparison

- **T version: 10 literals vs D version: 30 literals → T is much simpler.**
- **Why:** a counter's job is to **toggle** bits in a pattern. $T = 1$ encodes "this bit changes", which depends only on the lower bits and UD. D must encode the **new value**, which also depends on the bit's own current value ($D = Q \oplus T$), so every D map is the T map XORed with Q: no don't-cares and checkerboard-like maps. JK/T excitation tables suit counters; D's table has no freedom.

Circuit (T-based, the simpler one)

Gate equations

$$\begin{aligned} T_0 &= 1 \\ T_1 &= UD' \cdot Q_0 + UD \cdot Q_0' \quad (= UD \oplus Q_0 : \text{one XOR also works}) \\ T_2 &= UD' \cdot Q_1 \cdot Q_0 + UD \cdot Q_1' \cdot Q_0' \end{aligned}$$

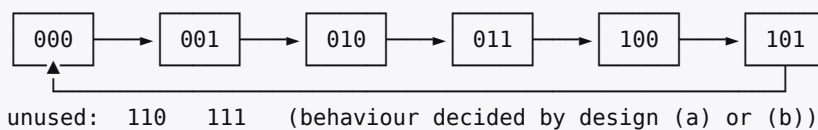


Key Logic & Takeaway

- Ripple counters: the edge type and the Q/Q' choice together decide up vs down.
- Synchronous counters: $T(i) = 1$ when all lower bits are 1 (up) or all lower bits are 0 (down). T/JK flip-flops give far less logic than D.

Tut-7 Q2 — Counters with Unused States (mod-6, JK flip-flops)

State diagram



Excitation table (JK: 0→0 = 0X, 0→1 = 1X, 1→0 = X1, 1→1 = X0)

Q2 Q1 Q0	Next (a)	J2 K2	J1 K1	J0 K0	Next (b)	J2 K2	J1 K1	J0 K0
000	001	0 X	0 X	1 X	001	0 X	0 X	1 X
001	010	0 X	1 X	X 1	010	0 X	1 X	X 1
010	011	0 X	X 0	1 X	011	0 X	X 0	1 X
011	100	1 X	X 1	X 1	100	1 X	X 1	X 1
100	101	X 0	0 X	1 X	101	X 0	0 X	1 X
101	000	X 1	0 X	X 1	000	X 1	0 X	X 1
110	XXX	X X	X X	X X	000	X 1	X 1	0 X
111	XXX	X X	X X	X X	000	X 1	X 1	X 1

(a) Unused states as don't-cares — K-maps (rows Q2, columns Q1Q0)

J2	00	01	11	10	K2	00	01	11	10
Q2=0	0	0	1	0	Q2=0	X	X	X	X
Q2=1	X	X	X	X	Q2=1	0	1	X	X

- $J_2 = Q_1Q_0$, $K_2 = Q_0$.

J1	00	01	11	10		K1	00	01	11	10
Q2=0	0	1	X	X		Q2=0	X	X	1	0
Q2=1	0	0	X	X		Q2=1	X	X	X	X

- $J1 = Q2'Q0$ (cannot be just $Q0$: $m5 = 0$), $K1 = Q0$.
- $J0 = 1$, $K0 = 1$ ($Q0$ toggles every clock; all specified cells are 1 or X).
- **Cost: 2 two-input AND gates (6 literals).**

(b) Unused states forced to 000

J2	00	01	11	10		K2	00	01	11	10
Q2=0	0	0	1	0		Q2=0	X	X	X	X
Q2=1	X	X	X	X		Q2=1	0	1	1	1

- $J2 = Q1Q0$, $K2 = Q1 + Q0$.

J1	00	01	11	10		K1	00	01	11	10
Q2=0	0	1	X	X		Q2=0	X	X	1	0
Q2=1	0	0	X	X		Q2=1	X	X	1	1

- $J1 = Q2'Q0$, $K1 = Q2 + Q0$.

J0	00	01	11	10		K0	00	01	11	10
Q2=0	1	X	X	1		Q2=0	X	1	1	X
Q2=1	1	X	X	0		Q2=1	X	1	1	X

- $J0 = Q2' + Q1' = (Q2Q1)'$ (one NAND), $K0 = 1$.
- **Cost: 2 ANDs + 2 ORs + 1 NAND (10 literals).**

Design (a)

$J2 = Q1 \cdot Q0$ $K2 = Q0$
 $J1 = Q2' \cdot Q0$ $K1 = Q0$
 $J0 = 1$ $K0 = 1$

Design (b)

$J2 = Q1 \cdot Q0$ $K2 = Q1 + Q0$
 $J1 = Q2' \cdot Q0$ $K1 = Q2 + Q0$
 $J0 = (Q2 \cdot Q1)'$ $K0 = 1$

$J0 \rightarrow$

$K0 \rightarrow$

$CLK \rightarrow$

J Q0

K FF0

>

$J1 \rightarrow$

$K1 \rightarrow$

$CLK \rightarrow$

J Q1

K FF1

>

$J2 \rightarrow$

$K2 \rightarrow$

$CLK \rightarrow$

J Q2

K FF2

>

all three FFs share CLK;
J/K come from the gates above,
fed by $Q0$, $Q1$, $Q2$ and $Q2'$

(c) What happens from 110 or 111?

Design (a) (don't-cares)

From	J2 K2 ($Q1Q0$, $Q0$)	J1 K1 ($Q2'Q0$, $Q0$)	J0 K0	Next
110	0, 0 $\rightarrow$ hold 1	0, 0 $\rightarrow$ hold 1	1, 1 $\rightarrow$ toggle 0 $\rightarrow$ 1	111
111	1, 1 $\rightarrow$ toggle $\rightarrow$ 0	0, 1 $\rightarrow$ reset $\rightarrow$ 0	1, 1 $\rightarrow$ toggle $\rightarrow$ 0	000 ✓

- 110 $\rightarrow$ 111 $\rightarrow$ 000: **2 clocks** to recover. 111 $\rightarrow$ 000: **1 clock**.
- It **does** recover (it is self-starting, by luck), but it passes through an invalid count (7) on the way.

Design (b) (explicit)

- 110 $\rightarrow$ 000 and 111 $\rightarrow$ 000: **1 clock** each, guaranteed by design.

(a) $110 \rightarrow 111 \rightarrow 000 \rightarrow 001 \dots$

(b) $110 \rightarrow 000 \rightarrow 001 \dots$
 $111 \rightarrow$

Trade-off

- Don't-care design: **cheaper** (6 vs 10 literals; no OR gates), but behaviour from the unused states is **whatever the minimised logic happens to do**. It must be checked by tracing. Here it recovers, but it could have locked into an unused loop (e.g. $110 \leftrightarrow 111$ forever) in another design.
- Explicit design: **costs more gates**, but recovery is guaranteed in one cycle.
- ⚠ EXAM TRICK: whenever you use unused states as don't-cares, **always** trace each unused state through your final equations and draw its arrows on the state diagram. "Self-starting" is something you show, not assume.

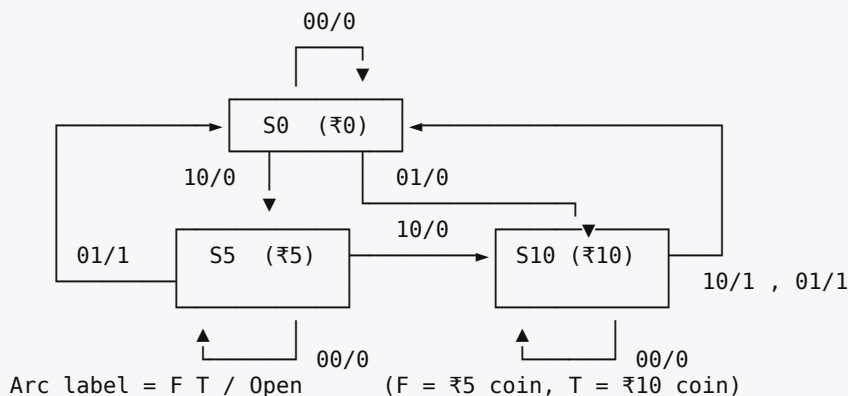
Key Logic & Takeaway

- Don't-cares minimise logic; explicit next states buy robustness.
- JK: read J from the Q = 0 half and K from the Q = 1 half of each map.

Tut-7 Q3 — Coin-Operated Turnstile

(a) State diagram, number of states, number of FFs

- Track the money accumulated so far: **S0 = ₹0, S5 = ₹5, S10 = ₹10**. Reaching $\geq ₹15$ opens the gate for one cycle and returns to S0.
- Mealy design**: Open is produced on the transition (it depends on the coin input). 3 states $\rightarrow \lceil \log_2 3 \rceil = 2$ flip-flops (one code left unused).
- Inputs: F (₹5), T (₹10), never both 1. Arc label FT/Open.



- Transition list (clearer than any drawing):

From	FT = 00 (no coin)	FT = 10 (₹5)	FT = 01 (₹10)
S0 (₹0)	S0 / 0	S5 / 0	S10 / 0
S5 (₹5)	S5 / 0	S10 / 0	S0 / 1 (₹15)
S10 (₹10)	S10 / 0	S0 / 1 (₹15)	S0 / 1 (₹20, change forgiven)

- Check the examples: $5+5+5 \rightarrow S5 \rightarrow S10 \rightarrow$ open on the 3rd coin ✓; $10+10 \rightarrow S10 \rightarrow$ open on the 2nd ✓; $5+10 \rightarrow S5 \rightarrow$ open on the 2nd ✓.
- Moore alternative**: add a 4th state S_OPEN (Open = 1 inside it, next state always S0). Still 2 FFs (no unused code), but the gate opens one cycle **after** the coin arrives.

(b) State assignment, JK excitation table, equations

- Codes: **S0 = 00**, **S5 = 01**, **S10 = 10**, unused **11**. Variables Q1 Q0 F T.

Q1 Q0	F T	Q1+ Q0+	J1 K1	J0 K0	Open
00	00	00	0 X	0 X	0
00	01	10	1 X	0 X	0
00	10	01	0 X	1 X	0
01	00	01	0 X	X 0	0
01	01	00	0 X	X 1	1
01	10	10	1 X	X 1	0
10	00	10	X 0	0 X	0
10	01	00	X 1	0 X	1
10	10	00	X 1	0 X	1
11	xx	XX	X X	X X	X
any	11	XX	X X	X X	X

- K-maps (rows Q1Q0, columns FT in the order 00, 01, 11, 10; column 11 and row 11 are all X):

J1	00	01	11	10	K1	00	01	11	10
00	0	1	X	0	00	X	X	X	X
01	0	0	X	1	01	X	X	X	X
11	X	X	X	X	11	X	X	X	X
10	X	X	X	X	10	0	1	X	1

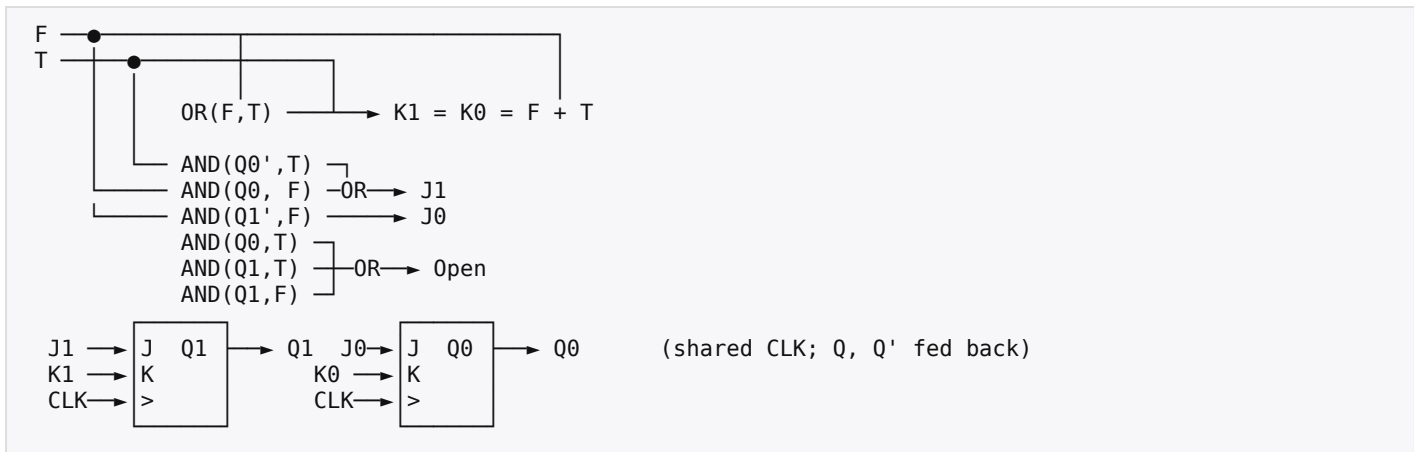
- J1 = Q0'T + Q0F**, **K1 = F + T**.

J0	00	01	11	10	K0	00	01	11	10
00	0	0	X	1	00	X	X	X	X
01	X	X	X	X	01	0	1	X	1
11	X	X	X	X	11	X	X	X	X
10	0	0	X	0	10	X	X	X	X

- J0 = Q1'F**, **K0 = F + T**.

Open	00	01	11	10
00	0	0	X	0
01	0	1	X	0
11	X	X	X	X
10	0	1	X	1

- Pls (all essential): **Q0T**, **Q1T**, **Q1F** → **Open = Q0T + Q1T + Q1F = T(Q1 + Q0) + Q1F**.



(c) The unused code 11

- **Choice made:** treated as don't-care (next state and Open) to minimise logic. It is never entered in normal operation, and $F = T = 1$ never happens.
- **Check what the logic actually does from 11** ($Q1 = Q0 = 1$):
 - No coin: $J1 = 0, K1 = 0 \rightarrow Q1$ holds; $J0 = 0, K0 = 0 \rightarrow Q0$ holds $\rightarrow$ **stays in 11**, Open = 0.
 - F: $J1 = Q0F = 1, K1 = 1 \rightarrow Q1$ toggles to 0; $J0 = 0, K0 = 1 \rightarrow Q0 = 0 \rightarrow$ **00**, Open = $Q1F = 1$.
 - T: $J1 = 0, K1 = 1 \rightarrow 0$; $J0 = 0, K0 = 1 \rightarrow 0 \rightarrow$ **00**, Open = 1.
- So 11 behaves **exactly like S10** (₹10 credit): it waits, then opens on any coin and returns to S0. It cannot lock up.
- **Risk:** a power-on glitch into 11 gives a free ₹10 credit. In practice add a reset to force 00 at power-up, or redesign $11 \rightarrow 00$ explicitly (more gates).

Key Logic & Takeaway

- Count the distinct "memories" the machine needs $\rightarrow$ states $\rightarrow \lceil \log_2 n \rceil$ flip-flops.
- Mealy = fewer states, output reacts in the same cycle; Moore = an extra state, output a cycle later but glitch-free.
- Always trace unused codes through the final equations.

BONUS — Past-Paper Sequential Problems (not register-based)

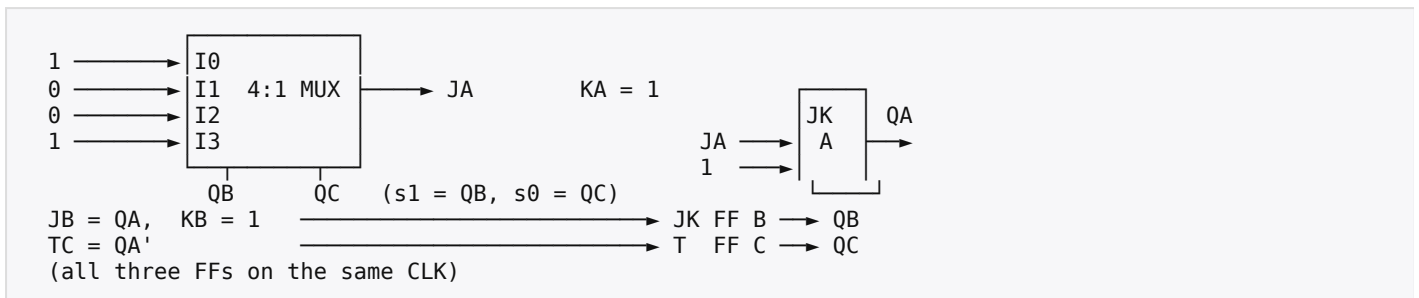
Two midsem FSM/counter questions in exactly the Tut-7 style, solved with the same method. Worth one read before the exam.

Midsem 2024-25 Q1 — Counter $4 \rightarrow 2 \rightarrow 1 \rightarrow 0 \rightarrow 5 \rightarrow 3 \rightarrow (4)$

- Components allowed: **one T FF, two JK FFs, one 4:1 mux**. Unused states (6, 7) are don't-cares.

QA QB QC	Next	JA KA	JB KB	TC
100 (4)	010	X 1	1 X	0
010 (2)	001	0 X	X 1	1
001 (1)	000	0 X	0 X	1
000 (0)	101	1 X	0 X	1
101 (5)	011	X 1	1 X	0
011 (3)	100	1 X	X 1	1
110, 111	X	X X	X X	X

- K-map results:
 - $JA = QB'QC' + QBQC = (QB \oplus QC)'$ (XNOR), $KA = 1$.
 - $JB = QA$, $KB = 1$.
 - $TC = QA'$ ($TC = 1$ in states 2, 1, 0, 3, all of which have $QA = 0$).
- Assignment of the parts:** the T FF for C ($TC = QA'$, a wire from QA'); JK for A and B; the **4:1 mux** makes the **XNOR**: selects QB, QC, data 1, 0, 0, 1.

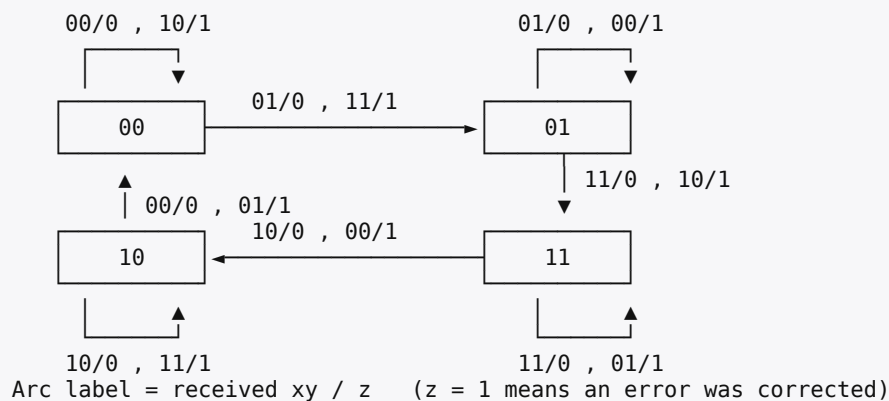


- Self-start check:** $110 \rightarrow (JA = 0, KA = 1 \rightarrow 0; JB = KB = 1 \rightarrow 0; TC = 0 \rightarrow 0) \rightarrow \mathbf{000} \checkmark$. $111 \rightarrow (JA = 1, KA = 1 \rightarrow \text{toggle} \rightarrow 0; B \rightarrow 0; C \text{ stays } 1) \rightarrow \mathbf{001} \checkmark$. Both enter the main cycle.

Midsem 2025-26 Q1 Task C — 2-bit Gray code error-correcting FSM

- The state = last correct angle code ($Q1 Q0$). The received code is $x y$ ($x = B1$). The true next code is either the same or the next Gray code ($00 \rightarrow 01 \rightarrow 11 \rightarrow 10 \rightarrow 00$). A received code that matches neither is corrected to the valid code at Hamming distance 1. Output $z = 1$ on error.

$Q1Q0 \setminus xy$	00	01	11	10
00	00/0	01/0	01/ 1	00/ 1
01	01/ 1	01/0	11/0	11/ 1
11	10/ 1	11/ 1	11/0	10/0
10	00/0	00/ 1	10/ 1	10/0



- **C2: which flip-flop?** Compute both:
 - D: $D1 = Q0x + Q1x + Q1Q0$, $D0 = Q0y + Q1'y + Q1'Q0 \rightarrow 12$ literals.
 - JK: $J1 = Q0x$, $K1 = Q0'x'$, $J0 = Q1'y$, $K0 = Q1y' \rightarrow 8$ literals ✓.
 - **Choose JK.** Its don't-cares halve each map.
- Output: $z = Q0'xy + Q0x'y' + Q1'xy' + Q1x'y$ (or the equivalent $Q1'Q0'x + Q1'Q0y' + Q1Q0'y + Q1Q0x'$). It is cyclic, with two equally minimal forms.

FINAL CHECKLIST FOR SEQUENTIAL QUESTIONS

- Use the **exact column order and names** the question gives ($D0/J0/K0$, $Q1$ $Q0$). Marking is often all-or-none per table.
- Excitation table first, then **one K-map per FF input**. Read J from the $Q = 0$ half and K from the $Q = 1$ half.
- Mealy vs Moore: output on arcs vs in states. State which one you built.
- Unused states: either make them don't-cares and **trace them**, or force them to a known state.
- Ripple counter direction = (edge type) $\times$ (Q or Q').
- Async set/reset acts immediately and overrides the clock; releasing it does nothing until the next edge.